

Multi-Drone Digital Array Radar Simulation

Overview

This MATLAB simulation demonstrates a **multi-drone detection and tracking system** using an **8×8 digital array radar** operating at **10 GHz**. The system showcases advanced radar signal processing including adaptive beamforming, micro-Doppler analysis, and simultaneous multi-target tracking.

Key Features

- ✓ **4 Drones** at randomly generated positions with independent trajectories
 - ✓ **8×8 Digital Array** (64 elements) with phase-shift beamforming
 - ✓ **Adaptive Steering** toward each drone dynamically
 - ✓ **Micro-Doppler Signatures** from rotating rotor blades
 - ✓ **Range-Doppler Processing** for target detection
 - ✓ **Beamforming Dynamics** visualization showing gain and pattern evolution
-

System Architecture

Hardware Configuration

RADAR SPECIFICATIONS

└ Carrier Frequency	: 10 GHz
└ Sample Rate	: 25 MHz
└ PRF (Pulse Rep. Freq)	: 5 kHz
└ Chirp Bandwidth	: 10 MHz
└ Range Resolution	: 15 m
└ Array Type	: 8×8 URA (uniform rectangular array)
└ Elements	: 64
└ Spacing	: $\lambda/2 = 1.5$ cm
└ Theoretical Gain	: 18.06 dB (coherent)

TRANSMITTER & RECEIVER

└ Tx Peak Power	: 1500 W
└ Tx Gain	: 25 dB
└ Rx Gain	: 25 dB
└ Rx Noise Figure	: 2 dB

Drone Configuration

Each drone simulates a quadcopter with:

- **1 Body scatterer** (RCS = 0.2 m²)
- **8 Blade-tip scatterers** (RCS = 0.01 m² each)
 - 4 rotors, 2 blades per rotor
 - Blade radius: 0.12 m
 - Rotation rate: 150 RPM (942 rad/s)

Position Generation: Randomly initialized within:

- X: 300–800 m (range from radar)

- Y: -200 to +200 m (lateral)
- Z: 80-150 m (altitude)

Velocity Generation: Randomized component-wise:

- $V_x, V_y: \pm 10 \text{ m/s}$
 - $V_z: \pm 2 \text{ m/s}$
-

Files Description

1. multi_drone_radar_main.m (Primary Simulation)

Purpose: Core radar simulation engine

Key Sections:

```
matlab
```

```
% Constants & Parameters
- Carrier frequency (10 GHz), wavelength, radar constants
- Array geometry (8x8 URA)
- Waveform (LFM chirp with 10 MHz bandwidth)

% Random Drone Initialization
- 4 drones with random positions and velocities
- Independent motion models using phased.Platform

% Main Pulse Loop (Niter iterations)
- Step 1: Get all drone kinematics (positions, velocities, angles)
- Step 2: Transmit waveform
- Step 3: Free-space propagation to all drones
- Step 4: Radar target reflections (RCS models)
- Step 5: Receive on 64 array elements
- Step 6: Digital beamforming (phase-shift steering)
- Step 7: SNR estimation

% Post-Processing
- Matched filtering (range compression)
- Group delay correction
- Range tracking
- Signal extraction and spectrograms

% Plotting & Analysis
- Multi-drone range tracks
- Radial velocity profiles
- Micro-Doppler spectrograms
- 3D flight paths
- Beamforming steering angle evolution
```

Outputs:

- **Figures:** 7 plots showing tracking and beamforming dynamics
- **Console:** Simulation progress, performance summary statistics

Key Parameters (User-Configurable):

```
matlab

Ndrones = 4;                % Number of drones
totalDuration = 10;         % Simulation time (seconds)
fc = 10e9;                  % Carrier frequency (10 GHz)
```

2. multiDroneMotion.m (Scatterer Kinematics)

Purpose: Generate drone body and blade-tip positions/velocities for one drone per call

Function Signature:

```
matlab

[scatterPos, scatterVel, scatterAng] = multiDroneMotion( ...
    t, dt, tgtmotion, radarpos, ...
    rotorOffset, rotorRadius, bladeRate, bladePhase)
```

Inputs:

- `t`: Current simulation time (seconds)
- `dt`: Time step for `tgtmotion` advancement (1/PRF)
- `tgtmotion`: `phased.Platform` object (drone body kinematics)
- `radarpos`: Radar position (3×1 vector)
- `rotorOffset`: Rotor hub offsets relative to body (3×4 matrix)
- `rotorRadius`: Blade length (meters)
- `bladeRate`: Rotor angular velocity (rad/s)
- `bladePhase`: Initial phase per rotor (4×1 vector)

Outputs:

- `scatterPos`: 3×9 positions (body + 8 blade tips)
- `scatterVel`: 3×9 velocities
- `scatterAng`: 2×9 azimuth/elevation angles from radar

Physics Modeled:

- **Body Motion:** Kinematic trajectory from `tgtmotion`
 - **Blade Rotation:** Circular rotor motion in x-y plane
 - **Blade Tip Velocity:** Body translation + tangential rotation component
 - **Look Angles:** Computed using `rangeangle()` function
-

3. beamforming_analysis.m (Advanced Beam Analysis)

Purpose: Detailed visualization and analysis of array beamforming performance

Key Analyses:

1. **Individual Beam Patterns** (Figure 1)
 - 4 plots showing radiation patterns steered toward each drone
 - Demonstrates narrowing and main-lobe gain
2. **3D Beam Patterns** (Figure 2)
 - Full 3D gain pattern for primary drone
 - Shows sidelobe structure and coverage
3. **Multi-Beam Gain Response** (Figure 3)
 - Overlay of beam responses across azimuth
 - Shows beam isolation capability
4. **Polar Radiation Patterns** (Figure 4)
 - Elevation slice patterns in polar form
 - Visualization of nulls and sidelobes

5. **Beamforming Gain Analysis** (Figure 5)

- On-axis vs off-axis gain comparison
- Sidelobe suppression metrics

6. **Spatial Resolution** (Figure 6)

- 3 dB beamwidth measurement
- Theoretical vs measured beamwidth
- Typical result: $\sim 3^\circ$ for 8×8 array at 10 GHz

7. **Beam Steering Dynamics** (Figure 7)

- Heatmap showing gain response as function of steer and scan angle
- Reveals beam shape and sidelobe behavior

8. **Adaptive Weights Visualization** (Figure 8)

- Magnitude and phase of beamformer weights per array element
- Shows phase taper from steering
- Shows weight tapering from element to edge

9. **Multi-Beam Simultaneous Tracking** (Figure 9)

- Approximate multi-beam capability
- Interference rejection at other drone positions

Usage:

```
matlab

% Run after multi_drone_radar_main.m completes
% Or run independently with fixed drone positions
>> beamforming_analysis
```

Outputs:

- 9 comprehensive figures with quantitative analysis

- Console: Beamforming performance metrics
-

Digital Beamforming Explanation

Phase-Shift Beamformer (Delay-and-Sum)

The core beamforming operation is:

$$\text{Beamformer Output} = \mathbf{w}^H \times \mathbf{y}$$

Where:

- $\mathbf{w}$ = steering vector (phase shift weights)
- $\mathbf{y}$ = received signals from all array elements
- H = conjugate transpose (Hermitian)

Steering Vector

For a URA with elements at positions (x_m, y_n) , the steering vector to angle (θ_z, θ_l) is:

$$w(\theta_z, \theta_l) = \exp[j \cdot 2\pi \cdot (x_m \cdot \sin(\theta_z) \cdot \cos(\theta_l) + y_n \cdot \sin(\theta_l)) / \lambda]$$

This phase-shift creates constructive interference in the desired direction and destructive interference elsewhere.

Key Advantages of Digital Arrays

Feature	Analog Sum	Digital Array
Beamforming Rate	Fixed (mechanical)	Per-pulse (electronic)
Multiple Beams	Single beam	Multiple simultaneous
Adaptive Steering	Slow	Fast (μs)
Element Data	Discarded	Preserved for processing
SNR Improvement	$\approx \sqrt{N_{\text{elements}}}$	$\approx N_{\text{elements}}$ (coherent)

Beamforming Gain

Coherent array gain: $10 \cdot \log_{10}(N)$ dB

For our 64-element array: **18.06 dB**

This represents the SNR improvement when all elements combine coherently (perfect phase alignment).

Simulation Workflow

Pulse-by-Pulse Processing

```
FOR each pulse m = 1 to Niter:
```

1. GET KINEMATICS

```
  For each drone d:
```

- Advance body position using phased.Platform(dt)
- Compute all blade-tip positions
- Calculate look angles [Az, El] from radar

2. TRANSMIT

- Generate LFM chirp pulse
- Apply transmitter gain
- Radiate from all array elements

3. PROPAGATE

- Free-space channel (path loss + propagation delay)
- Two-way propagation (Tx path + Rx path)

4. REFLECT

- Apply RCS model to each drone scatterer
- Body RCS: 0.2 m^2
- Blade RCS: 0.01 m^2 each

5. RECEIVE

- Collect signals on all 64 array elements
- Add receiver noise
- Preserve per-element data (key difference from analog sum)

6. BEAMFORM

- Compute steering vector for primary drone
- Combine 64 element signals with phase-shift weights:

```

        rxsig[m] = sum( w * element_signal )

7. ESTIMATE SNR
    - Compute signal power for each drone
    - Log for later analysis

END FOR

POST-PROCESSING:

8. MATCHED FILTER
    - Range compression using MF coefficients
    - Group delay correction

9. RANGE TRACKING
    - Find peak energy bin each pulse
    - Extract slow-time signal for Doppler processing

10. ANALYSIS
    - Micro-Doppler spectrogram
    - Range-Doppler maps
    - 3D trajectories

```

Simulation Time Estimate

- **Duration:** 10 seconds simulated time
 - **Pulses:** 50,000 (at 5 kHz PRF)
 - **Array Elements:** 64
 - **Samples/Pulse:** ~5,000 (at 25 MHz sample rate)
 - **Typical Runtime:** 5-15 minutes (depends on system)
-

Key Outputs & Interpretation

Plot 1: Range Tracking

Shows ground-truth range to each drone vs. matched-filter detection.

- **Expected:** Parallel tracks with varying slopes (different radial velocities)
- **X-axis:** Time (s)
- **Y-axis:** Range (m)
- **Color:** One per drone + detected range overlay

Plot 2: Radial Speed

Radial velocity component for each drone.

- **Positive:** Receding (moving away from radar)
- **Negative:** Approaching
- **Slope Changes:** Indicate acceleration/maneuver

Plot 3: Micro-Doppler Spectrogram

Time-frequency plot showing Doppler content of tracked signal.

- **Horizontal Lines:** Body Doppler (constant radial velocity periods)
- **Spreading/Modulation:** Blade rotation Doppler
- **Blade Flash Rate:** Spacing between modulation sidebands

Plot 4: SNR Evolution

Signal power over time per drone.

- **Peaks:** Drone closest/largest RCS orientation
- **Dips:** Drone receding or perpendicular orientation
- **Trend:** Overall SNR depends on range as $1/R^4$

Plot 5: 3D Flight Paths

Shows all drone trajectories in 3D space.

- **Radar:** Located at origin (or at $[0, 0, 50]$ m)
- **Circles:** Drone start positions
- **Squares:** Drone end positions
- **Lines:** Flight paths connecting them

Plot 6: Beamforming Angles

Steering azimuth and elevation angles over time.

- **Azimuth:** Azimuth beam center (-90° to $+90^\circ$)
- **Elevation:** Elevation beam center (typically 10° – 30°)
- **Dynamics:** Show how beam tracks primary target

Plot 7: Radiation Patterns

Antenna gain patterns at 4 time snapshots.

- **Narrow Main Lobe:** Directional gain in steered direction
- **Sidelobes:** Residual response at other angles
- **Width:** $\sim 3^\circ$ 3dB beamwidth typical for 8×8 array at 10 GHz

Advanced Features

1. Adaptive Beamforming Strategy

The simulation uses a **closest-drone steering policy**:

- Identify drone with minimum range each pulse
- Steer beam toward that drone's exact look angle
- Other drones appear in sidelobes (lower SNR)

Alternative Strategies (easily modified in code):

- **Scan-on-demand:** Rotate beam to scan each drone sequentially
- **Phased tracking:** Use Kalman filter to predict angles
- **Multi-beam:** Form 2–4 simultaneous beams via Eigen-decomposition

2. Micro-Doppler Exploitation

Each blade generates **sidebands** in frequency domain:

$$f_{\text{sideband}} = f_{\text{Doppler}} \pm k \times f_{\text{blade_flash}}$$

Where:

- f_{Doppler} = body radial velocity Doppler
- $k = 0, 1, 2, 3, \dots$ (sideband order)
- $f_{\text{blade_flash}} = 2 \times (\text{blade_rate} / 2\pi) \times (\text{number_of_blades})$

For our quadcopter: $f_{\text{blade_flash}} = 2 \times 150 \text{ RPM} \times 2 \text{ blades} / 60\text{s} = 10 \text{ Hz}$

Benefit: Micro-Doppler signature unique per drone type → classification

3. Range-Doppler Processing

Creates 2D map of range vs. radial velocity:

- **X-axis:** Radial velocity (Doppler frequency converted)
- **Y-axis:** Range (from matched filter output)
- **Z-axis:** Signal magnitude (or SNR)

Reveals:

- Precise range and velocity simultaneously
- Multiple targets at different ranges visible
- Clutter suppression via moving target indicator (MTI)

4. Array Gain & Sidelobe Control

Current implementation uses **uniform weighting** across elements.

Optional Enhancements:

- **Hamming/Blackman Window:** Reduce sidelobes by ~20 dB (cost: wider main lobe)
- **Chebyshev/Dolph-Tschebyscheff:** User-specified sidelobe level
- **Adaptive Nulling:** Place nulls at jammer positions

matlab

```
% Example: Apply Hamming window weights
w_base = phased.SteeringVector(ura, fc, steerAng);
w_taper = w_base .* hamming(Nelements);
w_final = w_taper / norm(w_taper);
```

Usage Instructions

Quick Start

matlab

```
% 1. Open MATLAB and navigate to simulation folder
cd /path/to/multi_drone_radar/

% 2. Run main simulation
multi_drone_radar_main

% 3. Wait for completion (5-15 min)
% 4. 7 figures automatically generated

% 5. (Optional) Run advanced beamforming analysis
beamforming_analysis

% 6. (Optional) Run detailed range-Doppler analysis
% [Use built-in range-Doppler functions on rxsig]
```

Parameter Customization

Edit `multi_drone_radar_main.m`:

```
matlab

% Line ~60: Number of drones
Ndrones = 4; % Change to 2, 3, 5, 6, etc.

% Line ~78: Simulation duration
totalDuration = 10; % seconds (shorter for quick tests)

% Line ~50: Carrier frequency
fc = 10e9; % 10 GHz (or 5e9 for 5 GHz, etc.)

% Line ~30: PRF and sampling
prf = 5e3; % Pulses per second
fs = 25e6; % Sample rate (Hz)

% Line ~73: Array size
ura.Size = [8 8]; % Change to [4 4], [16 16], etc.
```

Troubleshooting

Issue	Solution
Out of Memory	Reduce <code>totalDuration</code> , increase <code>prf</code> , or reduce array size
Slow Simulation	Reduce <code>totalDuration</code> to 2-3 seconds for testing
Blank Plots	Ensure <code>rxsig</code> computed correctly; check console warnings
Beamforming Analysis Errors	Run <code>multi_drone_radar_main.m</code> first to initialize variables

Physics & Theory

Free-Space Propagation Model

The channel applies:

- **Path Loss:** $L = (4\pi R/\lambda)^2$ (two-way)
- **Propagation Delay:** $\tau = 2R/c$
- **Phase Rotation:** $\exp(-j \cdot 2\pi \cdot f \cdot \tau)$

For 10 GHz at 500 m range:

- Path Loss: ~155 dB
- Delay: ~3.33 μ s
- Compensated by Tx gain (25 dB) + Rx gain (25 dB) + array gain (18 dB)

RCS (Radar Cross Section) Model

Body RCS = 0.2 m²: Typical for small quadcopter airframe **Blade RCS = 0.01 m² each:**

Thin, fast-rotating blades present low cross-section

Actual RCS varies with:

- Aspect angle
- Frequency
- Polarization
- Blade orientation

Our constant-RCS model is a simplification; real systems would use frequency-dependent and angle-dependent models.

Matched Filter Frequency Response

For LFM chirp of bandwidth B and duration T:

$$|H(f)| = T \cdot \text{sinc}(\pi \cdot B \cdot T \cdot (f/B))$$

The match filter pulse-compresses the wideband chirp to a narrow main lobe:

- **Range Resolution** = $c/(2B) = 1.5 \text{ m}$ (for 10 MHz BW)
 - **Compression Ratio** $\approx B \cdot T = 10^5$ (range profile gain)
-

Advanced Topics

Velocity Ambiguity (Folding)

Radial velocity is sampled at $\text{PRF} = 5 \text{ kHz}$, giving:

$$v_{\text{max}} = (\lambda \cdot \text{PRF}) / 2 = (3\text{cm} \times 5000) / 2 = 75 \text{ m/s}$$

Velocities beyond $\pm 75 \text{ m/s}$ will alias. Our drone speeds ($\sim 20 \text{ m/s}$) are well within range.

Mutual Coupling

Our URA model assumes **isolated elements** (no mutual coupling). Real arrays exhibit:

- Element-to-element impedance coupling
- Pattern distortion
- Gain loss (typically 1–3 dB)

Mitigation: Larger element spacing or decoupling networks.

Clutter Suppression

The matched filter processes **only tracked target range**, effectively:

- Suppresses ground clutter (different ranges)
- Removes sidelobe clutter (different angles via beamformer)
- Leaves only Doppler content from target motion

True **MTI (Moving Target Indicator)** would further separate moving targets from stationary clutter via Doppler filtering.

References & Further Reading

MATLAB Documentation

- `phased.URA` - Uniform Rectangular Array
- `phased.SteeringVector` - Steering vector generation
- `phased.RangeDopplerResponse` - Range-Doppler processing
- `phased.LinearFMWaveform` - LFM chirp generation

Theory

- **Radar Basics:** "Introduction to Radar Systems" - M. Skolnik (MIT Press)
 - **Array Processing:** "Detection, Estimation, and Modulation Theory" - H. Van Trees
 - **Beamforming:** "Adaptive Filter Theory" - S. Haykin (Wiley)
 - **Micro-Doppler:** "Micro-Doppler Effect in Radar" - V. Chen & R. Lipps (Artech House)
-

Citation & Contact

Simulation Author: Generated for Multi-Drone Radar Education

Toolbox Version: MATLAB R2023b+ (Phased Array System Toolbox)

License: Educational Use

For questions or modifications, refer to inline code comments and MATLAB help.

Last Updated: 2026

Status: Ready for Educational & Research Use